

به نام خداوند جان و خرد



طراحی کامپیوتری سیستم‌های دیجیتال

گزارش پروژه نهایی

استاد درس :

آقای دکتر حاتم عبدلی

ارائه دهنده :

آرمان جهانگیریان - محمدحسین گل محمدی - نیمامخملی

40212358035

40212358034

40212358015

تابستان 1405

## VHDL

### VHSIC Hardware Description Language

#### با چه چیزی سروکار داشتیم ؟

در این پروژه یک شتاب‌دهنده سخت‌افزاری برای یک شبکه عصبی چندلایه (MLP) با هدف تشخیص ارقام دست‌نویس طراحی و پیاده‌سازی شده است.

مسیر طراحی از آموزش مدل نرم‌افزاری روی MNIST آغاز شده و پس از کاهش ابعاد تصاویر از  $28 \times 28$  به  $8 \times 8$ ، شبکه‌ای با ساختار 64 ورودی، 16 نورون در لایه پنهان و 10 خروجی آموزش داده شده است.

پارامترهای مدل استخراج و به نمایش Fixed-Point تبدیل شده‌اند و سپس معماری RTL شبکه با VHDL شامل واحد MAC، تابع ReLU، حافظه‌های ROM، رجیسترهای نتایج و کنترل‌کننده FSM پیاده‌سازی شده است.

برای اعتبارسنجی، Testbench تصاویر را از فایل خوانده و خروجی سخت‌افزار را با Prediction پیتون و Label واقعی مقایسه می‌کند. در ادامه طراحی در Vivado از نظر Timing، منابع سخت‌افزاری و توان قابل ارزیابی است.

همچنین چهار بخش امتیازی شامل Parallelization، Pipelining، Mixed-Precision و Approximate Computing پیاده‌سازی شده‌اند.

در Mixed-Precision، وزن‌ها از  $Q(8,8)$  شانزده‌بیتی به  $Q(4,4)$  هشت‌بیتی کاهش یافته‌اند و Accuracy گزارش‌شده از 91.14 % به 91.13 % رسیده است؛ یعنی افت دقت تنها 0.01 واحد درصد بوده است.

#### هدف ما از این پروژه چی بود ؟

شتاب‌دهنده‌های سخت‌افزاری نقش مهمی در اجرای سریع و کم‌مصرف الگوریتم‌های یادگیری ماشین دارند. شبکه‌های عصبی از عملیات ضرب و جمع متعددی تشکیل شده‌اند و همین ویژگی آن‌ها را برای پیاده‌سازی روی معماری‌های سخت‌افزاری موازی و قابل یکپارچگی مناسب می‌کند. هدف این پروژه، طی کردن یک مسیر کامل از مدل نرم‌افزاری تا معماری سخت‌افزاری یک شبکه عصبی ساده و بررسی اثر انتخاب‌های معماری بر دقت، Latency، منابع و توان است.

اهداف عملی پروژه شامل آموزش MLP، کاهش پیچیدگی ورودی، استخراج Bias و Weight، Quantization، طراحی MAC و ReLU، ذخیره پارامترها در ROM، طراحی FSM، ساخت Testbench، مقایسه Python و VHDL، سنتز و تحلیل در Vivado و پیاده‌سازی بخش‌های امتیازی است.

#### چه ساختاری داشتیم ؟

زنجیره طراحی پروژه به صورت زیر است:

MNIST → Preprocessing → MLP Training → Weight/Bias Extraction  
→ Fixed-Point Quantization → VHDL RTL → Simulation/Testbench  
→ Vivado Synthesis & Implementation → Timing/Resource/Power Analysis

## فاز اول: طراحی و آموزش مدل نرم‌افزاری

### دیتاست MNIST

برای آموزش شبکه عصبی از دیتاست معروف MNIST استفاده شده است. این مجموعه داده شامل تصاویر ارقام دست‌نویس صفر تا نه است.

تصاویر اصلی MNIST دارای ابعاد:

$28 \times 28$

هستند و هر تصویر شامل 784 پیکسل است.

اگرچه این ابعاد برای اجرای نرم‌افزاری مشکلی ایجاد نمی‌کند، استفاده مستقیم از 784 ورودی در یک شتاب‌دهنده سخت‌افزاری ساده باعث افزایش قابل توجه تعداد ضرب‌کننده‌ها، حافظه مورد نیاز و پیچیدگی طراحی خواهد شد.

به همین دلیل، پیش از ورود داده به شبکه، مرحله کاهش ابعاد انجام شده است.

## کاهش ابعاد تصاویر

در این پروژه تصاویر از ابعاد:

$28 \times 28$  به  $8 \times 8$

تبدیل شده‌اند.

بنابراین تعداد ویژگی‌های ورودی از:

784 به 64

کاهش یافته است.

در فرآیند preprocessing ابتدا تصویر Resize شده و سپس به Tensor تبدیل شده است. مقادیر پیکسل‌ها نیز در بازه  $[0,1]$  قرار گرفته‌اند. این preprocessing برای داده‌های آموزش و تست به صورت یکسان انجام شده است.

این کاهش ابعاد یک Trade-off مهم ایجاد می‌کند:

- کاهش قابل توجه پیچیدگی سخت‌افزار
- کاهش تعداد MAC مورد نیاز
- کاهش حافظه مورد نیاز
- افزایش سرعت شبیه‌سازی

در مقابل، بخشی از اطلاعات فضایی تصویر از بین می‌رود و ممکن است دقت طبقه‌بندی کاهش یابد.

## معماری شبکه عصبی

معماری انتخاب‌شده یک MLP ساده با یک لایه پنهان است:

$10 \rightarrow 16 \rightarrow 64$

بنابراین:

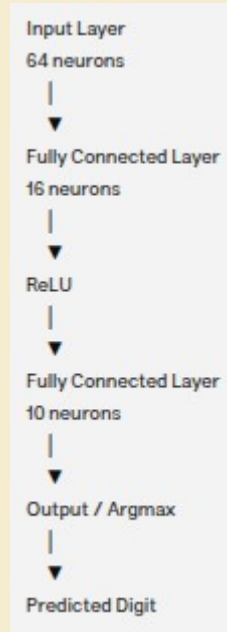
- تعداد ورودی‌ها: 64
- تعداد نوروهای لایه پنهان: 16
- تعداد خروجی‌ها: 10

ده خروجی متناظر با کلاس‌های:

9,...,0,1,2

هستند.

معماری شبکه به صورت زیر است :



این معماری مستقیماً در مدل Python پیاده‌سازی شده است و خروجی نهایی شبکه به صورت Logit تولید می‌شود. در مرحله inference به جای اعمال Softmax، بزرگ‌ترین Logit انتخاب می‌شود که همان عملیات Argmax است. این روش با معماری سخت‌افزار نیز کاملاً سازگار است.

## تابع فعال‌سازی ReLU

تابع فعال‌سازی استفاده‌شده در لایه پنهان ReLU است:

$$ReLU(x) = \max(0, x)$$

بنابراین:

$$ReLU(x) = \begin{cases} x & x \geq 0 \\ 0 & x < 0 \end{cases}$$

انتخاب ReLU برای این پروژه مناسب است، زیرا پیاده‌سازی سخت‌افزاری آن بسیار ساده است و می‌توان آن را با یک مقایسه‌کننده و Multiplexer پیاده‌سازی کرد.

## فرآیند آموزش

مدل با استفاده از PyTorch آموزش داده شده است.

پارامترهای اصلی آموزش عبارت‌اند از:

| پارامتر       | مقدار         |
|---------------|---------------|
| Batch Size    | 64            |
| Epoch         | 10            |
| Learning Rate | 0.01          |
| Optimizer     | SGD           |
| Momentum      | 0.9           |
| Random Seed   | 42            |
| Loss Function | Cross Entropy |

این مقادیر در بیکربندی پروژه مشخص شده‌اند.

در هر Epoch، مدل روی داده‌های آموزشی آموزش داده شده و سپس Accuracy روی مجموعه تست محاسبه شده است. وزن‌های نهایی مدل نیز در قالب Checkpoint ذخیره شده‌اند.

شکل پیشنهادی برای گزارش:

تصویر نمودار Loss و Accuracy در طول Epoch‌ها.

## استخراج Bias و Weight

پس از پایان آموزش، پارامترهای شبکه استخراج شده‌اند.

برای لایه پنهان:

$$W_H \in \mathbb{R}^{16 \times 64}$$

$$B_H \in \mathbb{R}^{16}$$

برای لایه خروجی:

$$W_O \in \mathbb{R}^{10 \times 16}$$

$$B_O \in \mathbb{R}^{10}$$

پارامترها در فایل‌های متنی ذخیره شده‌اند تا در مرحله Quantization و سپس تولید ROM‌های VHDL استفاده شوند. این روش باعث می‌شود یک مسیر داده یکپارچه از مدل نرم‌افزاری تا سخت‌افزار ایجاد شود.

## فاز دوم: Quantization و Fixed-Point

### ضرورت استفاده از Fixed-Point

محاسبات Floating-Point در سخت‌افزار نسبت به محاسبات Integer و Fixed-Point پیچیده‌تر هستند و معمولاً منابع سخت‌افزاری بیشتری نیاز دارند.

از آنجا که شتاب‌دهنده طراحی شده برای FPGA در نظر گرفته شده است، استفاده از Fixed-Point باعث کاهش پیچیدگی محاسبات و امکان استفاده بهتر از منابع FPGA می‌شود.

در این پروژه برای داده‌های اصلی از نمایش:

$$Q(8,8)$$

استفاده شده است.

### فرمت Q(8,8)

در فرمت Q(8,8) :

- 8 بیت برای بخش صحیح، شامل بیت علامت
- 8 بیت برای بخش اعشاری

در نظر گرفته شده است.

بنابراین طول کل داده:

$$8 + 8 = 16 \text{ bit}$$

است.

Scale Factor برابر است با:

$$2^8 = 256$$

بنابراین تبدیل یک مقدار اعشاری  $x$  به مقدار Fixed-Point به صورت:

$$x_{fixed} = round(x \times 256)$$

انجام می‌شود.

و برای تبدیل مجدد:

$$x \approx \frac{x_{fixed}}{256}$$

استفاده می‌شود.

## کوانتیزاسیون ورودی

مقادیر بیکسل تصاویر پس از preprocessing در بازه:

$$[0,1]$$

قرار دارند.

این مقادیر به فرمت  $Q(8,8)$  تبدیل شده و سپس به صورت Signed Integer در اختیار سخت‌افزار قرار می‌گیرند.

به عنوان مثال:

$$128 = 256 \times 0.5$$

بنابراین:

$$128 \rightarrow 0.5$$

در نمایش Fixed-Point تبدیل می‌شود.

## کوانتیزاسیون Bias و Weight

تمام Weight ها و Bias های مدل نیز به Fixed-Point تبدیل شده‌اند.

در نتیجه داده‌های مورد استفاده در معماری پایه به شکل زیر هستند:

| نوع داده      | قالب                                 |
|---------------|--------------------------------------|
| Input         | $Q(8,8)$ / 16-bit                    |
| Hidden Weight | $Q(8,8)$ / 16-bit                    |
| Hidden Bias   | $Q(8,8)$ / 16-bit                    |
| Output Weight | $Q(8,8)$ / 16-bit                    |
| Output Bias   | $Q(8,8)$ / 16-bit                    |
| Accumulator   | عرض بزرگ‌تر برای جلوگیری از Overflow |

مقادیر Quantized شده سپس برای تولید ROM های VHDL استفاده شده‌اند.

## شبیه‌سازی نرم‌افزاری محاسبات Fixed-Point

پیش از انتقال الگوریتم به VHDL، رفتار Fixed-Point در Python شبیه‌سازی شده است.

برای هر نورون ابتدا حاصل ضرب:

$$X_i \times W_i$$

محاسبه و سپس تمام حاصل ضرب‌ها با یکدیگر جمع می‌شوند:

$$S = \sum_i x_i w_i$$

سپس Bias اضافه می‌شود:

$$\hat{S} = S + b$$

در نهایت برای بازگرداندن نتیجه به مقیاس Q(8,8)، عملیات Rescale انجام می‌شود. این مرحله اهمیت زیادی دارد، زیرا باعث می‌شود رفتار مدل Fixed-Point پیش از پیاده‌سازی RTL مشخص باشد.

## بررسی دقت مدل

یکی از الزامات اصلی پروژه بررسی تفاوت Accuracy مدل Floating-Point و مدل Quantized است.

در پروژه این دو مدل به صورت مستقل روی داده‌های تست ارزیابی شده‌اند. نتایج باید به صورت زیر در گزارش ارائه شوند:

| مدل                   | Accuracy                  |
|-----------------------|---------------------------|
| Floating-Point        | % [عدد Accuracy]          |
| Fixed-Point<br>Q(8,8) | % [عدد Accuracy]          |
| Accuracy Drop         | percentage [عدد]<br>point |

فرمول افت دقت:

$$\text{AccuracyDrop} = \text{AccuracyFloat} - \text{AccuracyFixed}$$

است.

نکته: مقدار واقعی را باید از خروجی اجرای پروژه خودت وارد این جدول کنی.

## بررسی فرمت‌های مختلف Fixed-Point

برای بررسی انتخاب Q(8,8)، فرمت‌های مختلف نیز مورد آزمایش قرار گرفته‌اند.

این بررسی شامل قالب‌هایی مانند:

- Q(4,4)
- Q(12,4)
- Q(8,8)
- Q(4,12)
- Q(16,16)

است.

هدف این آزمایش بررسی رابطه میان:

Bit Width



Accuracy



Hardware Cost

است.

این بررسی نشان می‌دهد که انتخاب Fixed-Point نباید صرفاً بر اساس بیشترین دقت انجام شود، بلکه باید دقت، محدوده عددی، منابع سخت‌افزاری و هزینه محاسبات هم‌زمان در نظر گرفته شوند.

## فاز سوم: طراحی سخت‌افزار در VHDL

### معماری سخت‌افزار

معماری اصلی سخت‌افزار بر اساس یک طراحی Sequential و Resource-Sharing ساخته شده است.

در این معماری به جای استفاده از یک MAC مستقل برای هر نورون، یک MAC مشترک بین نورون‌ها استفاده می‌شود.

مزیت اصلی این معماری کاهش مصرف منابع است و در مقابل Latency افزایش پیدا می‌کند. کد top-level نیز صراحتاً معماری Shared MAC را مشخص کرده است.

## واحد MAC

واحد MAC مهم‌ترین بخش محاسباتی شبکه است.

وظیفه این واحد محاسبه:

$$ACC \leftarrow ACC + (A \times B)$$

در هر Clock Cycle است.

ساختار واحد MAC شامل:

- Multiplier
- Accumulator Register
- Control Signals

است.

در صورت فعال بودن enable، حاصل ضرب دو ورودی محاسبه شده و به Accumulator اضافه می‌شود. سیگنال clear\_acc نیز برای شروع محاسبه یک نورون جدید استفاده می‌شود.

## واحد ReLU

تابع ReLU در سخت‌افزار وظیفه حذف مقادیر منفی را بر عهده دارد.

از نظر منطقی:

در صورت مثبت بودن ورودی، همان مقدار عبور می‌کند و در صورت منفی بودن، صفر تولید می‌شود.

در معماری نهایی، این عملیات از طریق تابع  $\text{relu}$  در منطق شبکه اعمال شده و یک واحد ReLU مستقل نیز برای طراحی و تست سطح واحد در پروژه وجود دارد.

## حافظه‌های ROM

Bias و Weight در حافظه‌های ROM قرار گرفته‌اند.

ROM‌های اصلی عبارت‌اند از:

1. Hidden Weights ROM
2. Hidden Bias ROM
3. Output Weights ROM
4. Output Bias ROM

برای Hidden Layer تعداد وزن‌ها برابر است با:

$$1024 = 64 \times 16$$

و برای Output Layer:

$$160 = 16 \times 10$$

بنابراین مجموع Weight‌های شبکه:

$$1184 = 1024 + 160$$

مقدار است.

این پارامترها مستقیماً از خروجی مرحله Quantization به VHDL تبدیل شده‌اند.

## رجیسترهای لایه‌ها

برای ذخیره نتایج میانی از رجیسترها استفاده شده است.

مهم‌ترین آن‌ها عبارت‌اند از:

- Input Register

- Hidden Layer Register
- Output Register

ابتدا تصویر ورودی در رجیستر ورودی ذخیره می‌شود. سپس خروجی 16 نرون Hidden به صورت مرحله‌ای محاسبه و در hidden\_reg ذخیره می‌شود. بعد از آن این مقادیر به عنوان ورودی لایه خروجی استفاده می‌شوند.

## Rescaling

یکی از مسائل مهم در محاسبات Fixed-Point، حفظ مقیاس عددی است.

اگر:  $x$

و:  $w$

هر دو  $Q(8,8)$  باشند، حاصل ضرب آن‌ها دارای مقیاس:

$$216 = 28 \times 28$$

خواهد بود.

بنابراین پس از Accumulation باید نتیجه به مقیاس موردنظر برگردانده شود.

این کار با عملیات Shift و Rescale انجام شده است.

این مرحله برای جلوگیری از خطای عددی و هماهنگی ماندن محاسبات Python و VHDL بسیار مهم است.

## واحد کنترل و FSM

برای کنترل کامل جریان داده‌ها از یک Finite State Machine استفاده شده است.

State‌های اصلی طراحی عبارت‌اند از:

S\_IDLE

S\_H\_INIT  
S\_H\_MAC  
S\_H\_BIAS  
S\_H\_RESCALE  
S\_H\_NEXT

S\_O\_INIT  
S\_O\_MAC  
S\_O\_BIAS  
S\_O\_RESCALE  
S\_O\_NEXT

S\_ARGMAX\_INIT  
S\_ARGMAX\_STEP

S\_DONE

این ساختار در کد top-level تعریف شده است.

## محاسبه لایه پنهان

برای هر نورون Hidden ابتدا Accumulator صفر می‌شود.

سیس 64 ورودی به ترتیب با Weight 64 مربوط به نورون ضرب می‌شوند:

$$S_j = \sum_{i=0}^{63} x_i w_{j,i}$$

بعد Bias مربوط به همان نورون اضافه می‌شود:

$$z_j = S_j + b_j$$

سیس عملیات Rescale و ReLU انجام می‌شود:

$$h_j = \text{ReLU}(z_j)$$

این عملیات برای 16 نورون تکرار می‌شود.

## محاسبه لایه خروجی

خروجی لایه Hidden شامل 16 مقدار است.

برای هر نورون خروجی:

$$y_k = \sum_{j=0}^{15} h_j w_{k,j} + b_k$$

محاسبه می‌شود.

در طراحی سخت‌افزاری نیز Weight‌های خروجی و Hidden Register به Shared MAC متصل می‌شوند.

کد FSM این فرآیند را به صورت State‌های S\_O\_INIT، S\_O\_MAC، S\_O\_BIAS و S\_O\_RESCALE کنترل می‌کند.

## عملیات Argmax

در انتهای محاسبات، 10 خروجی شبکه در output\_reg ذخیره شده‌اند.

برای تعیین کلاس نهایی، خروجی‌ها با یکدیگر مقایسه می‌شوند:

$$class = \arg \max_k (y_k)$$

FSM ابتدا خروجی کلاس صفر را به عنوان مقدار اولیه در نظر گرفته و سپس سایر 9 خروجی را یکی یکی مقایسه می‌کند.

در پایان:

digit\_out = index of maximum output

قرار می‌گیرد و سیگنال done فعال می‌شود.

## فاز چهارم: شبیه‌سازی و اعتبارسنجی

### Testbench

برای اعتبارسنجی سخت‌افزار یک Testbench طراحی شده است.

Testbench وظیفه دارد:

1. فایل تصاویر تست را بخواند.

2. Label واقعی را بخواند.
3. Prediction مدل Python را بخواند.
4. تصویر Quantized را به ورودی شبکه بدهد.
5. منتظر فعال شدن done بماند.
6. Prediction سخت‌افزار را دریافت کند.
7. نتیجه را با Python مقایسه کند.
8. نتیجه را با Label واقعی مقایسه کند.
9. Latency را محاسبه کند.

این فرآیند باعث می‌شود تست فقط به بررسی یک خروجی Waveform محدود نباشد و اعتبار عملکرد شبکه نیز بررسی شود.

## نحوه اعمال ورودی‌ها

تصاویر تست به صورت فایل متنی ذخیره شده‌اند.

هر تصویر شامل 64 مقدار Quantized است.

Testbench این مقادیر را خوانده و در:

pixel\_in

قرار می‌دهد.

برای هر نمونه سه مقدار اصلی مورد استفاده قرار می‌گیرد:

Input Image

True Label

Python Prediction

## مقایسه Python و VHDL

برای هر نمونه دو مقایسه انجام شده است:

### مقایسه اول

Hardware Prediction=?Python Prediction

## مقایسه دوم

Hardware Prediction=?True Label

به این ترتیب می‌توان هم درستی انتقال الگوریتم Python به سخت‌افزار و هم دقت واقعی طبقه‌بندی را بررسی کرد.

## بررسی Waveform

خروجی شبیه‌سازی به صورت VCD تولید شده و با ابزار GTKWave قابل مشاهده است.

در Waveform باید سیگنال‌های زیر مورد بررسی قرار گیرند:

- clk
- rst
- start
- done
- state
- neuron\_idx
- elem\_idx
- mac\_en
- mac\_clear
- mac\_acc
- digit\_out

محل قرار دادن تصویر:

شکل 1-7 – Waveform مربوط به اجرای یک نمونه ورودی در شبیه‌ساز

در تحلیل Waveform مشاهده می‌شود که پس از فعال شدن FSM، start وارد مرحله محاسبه Hidden Layer شده و پس از پایان محاسبات Hidden، وارد Output Layer و سپس Argmax می‌شود. در نهایت با رسیدن FSM به S\_DONE، سیگنال done فعال و نتیجه در digit\_out قرار می‌گیرد.

## محاسبه Latency

Latency بر اساس تعداد Clock Cycle‌های مورد نیاز از زمان آغاز پردازش تا فعال شدن done محاسبه شده است.

با فرض:

$f_{clk}=100\text{MHz}$

دوره Clock برابر است با:

$T_{clk}=10\text{ns}$

بنابراین اگر یک نمونه در N سیکل پردازش شود:

$\text{Latency}=N \times 10\text{ns}$

خواهد بود.

در معماری پایه، به دلیل استفاده از Shared MAC، محاسبه نورهن‌ها به صورت Sequential انجام می‌شود. در نتیجه تعداد Cycle‌ها نسبتاً زیاد است؛ اما در مقابل منابع سخت‌افزاری کاهش پیدا می‌کنند. ساختار کد نیز همین Trade-off بین Resource و Throughput را هدف قرار داده است.

## پیاده‌سازی و تحلیل در Vivado

پس از اطمینان از صحت عملکرد RTL در شبیه‌سازی، طراحی در محیط Vivado سنتز و پیاده‌سازی شده است.

فرآیند کلی شامل:

VHDL Sources  
↓  
Synthesis  
↓  
Optimization  
↓  
Place & Route  
↓  
Timing Analysis  
↓  
Utilization Report  
↓  
Power Analysis

است.

## Synthesis

در مرحله Synthesis کد VHDL به یک Netlist سخت‌افزاری تبدیل شده است.

هدف این مرحله بررسی صحت سنتز و مشخص شدن منابع مورد استفاده توسط طراحی است.

## Timing Analysis

در Timing Analysis پارامترهای زیر بررسی شده‌اند:

- Clock Period
- Worst Negative Slack
- Total Negative Slack
- Maximum Frequency
- Setup Timing
- Hold Timing

نتایج اجرای Vivado:

| پارامتر          | نتیجه       |
|------------------|-------------|
| Clock Period     | [مقدار] ns  |
| Target Frequency | [مقدار] MHz |
| Achieved Fmax    | [مقدار] MHz |
| WNS              | [مقدار] ns  |
| TNS              | [مقدار] ns  |
| Timing Status    | [PASS/FAIL] |